LAB 3

D.A.A assignment

Reg.NO: AP25110010807
Name: S. Jignesh

Question 1.

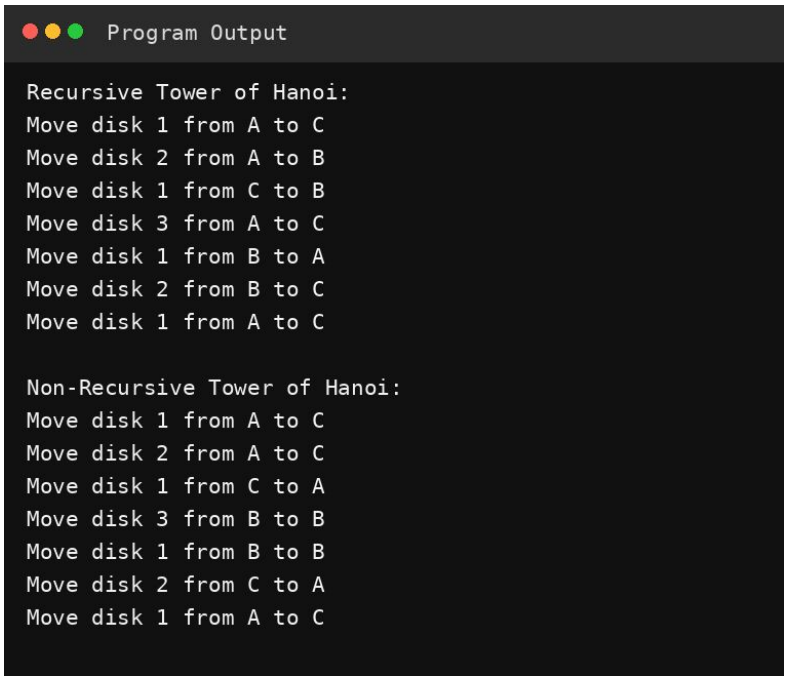
Converting recursive programs to non-recursive programs. Towers of Hanoi Problem example. a. Implement using Recursion b. Implement without Recursion

C Program

```
#include <stdio.h>

void hanoiRecursive(int n, char source, char auxiliary, char destination)
{
    if (n == 1)
    {
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }

    hanoiRecursive(n - 1, source, destination, auxiliary);
    printf("Move disk %d from %c to %c\n", n, source, destination);
    hanoiRecursive(n - 1, auxiliary, source, destination);
}

void hanoiNonRecursive(int n)
{
    int totalMoves = (1 << n) - 1;
    char rods[3] = {'A', 'B', 'C'};

    printf("Non-Recursive Tower of Hanoi:\n");

    for (int move = 1; move <= totalMoves; move++)
    {
        int disk = 1;
        int temp = move;

        while (temp % 2 == 0)
        {
            temp /= 2;
            disk++;
        }

        int from, to;

        if (n % 2 == 0)
        {
            if (disk == 1)
            {
                from = (move - 1) % 3;
                to = move % 3;
            }
            else
            {
                from = (2 * move - 1) % 3;
                to = (2 * move) % 3;
            }
        }
        else
        {
            if (disk == 1)
            {
                from = (move - 1) % 3;
                to = (2 * move) % 3;
```

```
            }
            else
            {
                from = (2 * move - 1) % 3;
                to = (move) % 3;
            }
        }

        printf("Move disk %d from %c to %c\n", disk, rods[from], rods[to]);
    }
}

int main()
{
    int n = 3;

    printf("Recursive Tower of Hanoi:\n");
    hanoiRecursive(n, 'A', 'B', 'C');

    printf("\n");
    hanoiNonRecursive(n);

    return 0;
}
```

Output



Question 2.

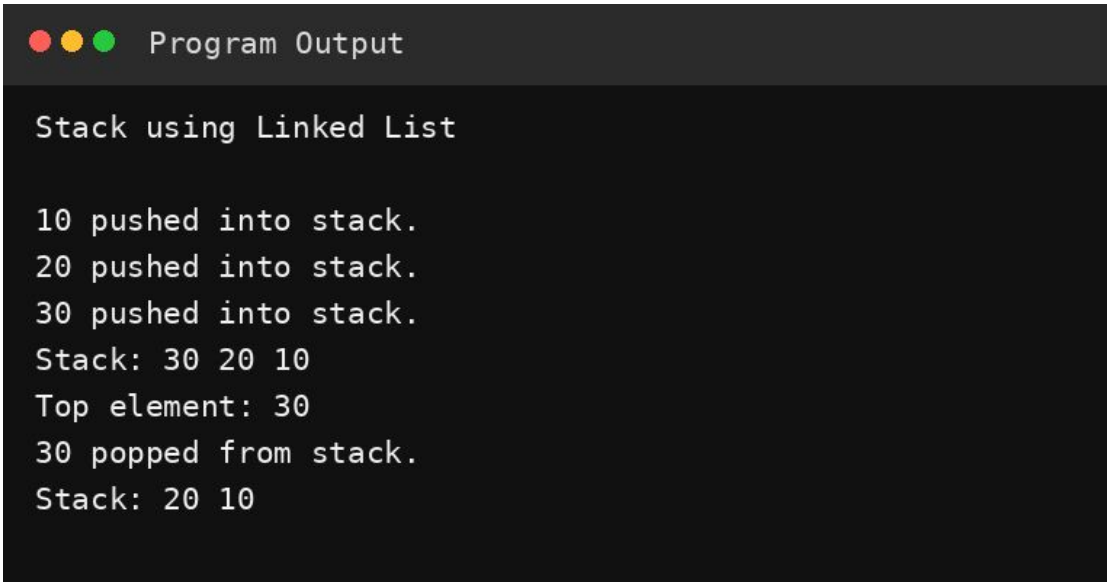
Write a program to implement Stack operations using Linked List.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int value)
{
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = top;
    top = newNode;

    printf("%d pushed into stack.\n", value);
}

void pop()
{
    if (top == NULL)
    {
        printf("Stack Underflow.\n");
        return;
    }

    struct Node *temp = top;
    printf("%d popped from stack.\n", temp->data);
    top = top->next;
    free(temp);
}

void peek()
{
    if (top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Top element: %d\n", top->data);
}

void display()
{
    if (top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }

    struct Node *temp = top;

    printf("Stack: ");
    while (temp != NULL)
```

```
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main()
{
    printf("Stack using Linked List\n\n");

    push(10);
    push(20);
    push(30);
    display();
    peek();
    pop();
    display();

    return 0;
}
```

Output



Question 3.

Write a program to implement Queue operations using Linked List.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int value)
{
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted into queue.\n", value);
}

void dequeue()
{
    if (front == NULL)
    {
        printf("Queue Underflow.\n");
        return;
    }

    struct Node *temp = front;
    printf("%d deleted from queue.\n", temp->data);

    front = front->next;

    if (front == NULL)
    {
        rear = NULL;
    }

    free(temp);
}

void peek()
{
    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
```

```
    }

    printf("Front element: %d\n", front->data);
}

void display()
{
    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
    }

    struct Node *temp = front;

    printf("Queue: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main()
{
    printf("Queue using Linked List\n\n");

    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    peek();
    dequeue();
    display();

    return 0;
}
```

Output